

请记住

- C++ 标准程序库的主要机能由 STL、iostreams、locales 组成。并包含 C99 标准程序库。
- TR1 添加了智能指针（例如 `tr1::shared_ptr`）、一般化函数指针（`tr1::function`）、hash-based 容器、正则表达式（regular expressions）以及另外 10 个组件的支持。
- TR1 自身只是一份规范。为获得 TR1 提供的好处，你需要一份实物。一个好的实物来源是 Boost。

条款 55: 让自己熟悉 Boost

Familiarize yourself with Boost.

你正在寻找一个高质量、源码开放、平台独立、编译器独立的程序库吗？看看 Boost 吧。有兴趣加入一个由雄心勃勃充满才干的 C++ 开发人员组成的社群，致力发展（设计和实现）当前最高技术水平之程序库吗？看看 Boost 吧！想要一瞥未来的 C++ 可能长相吗？看看 Boost 吧！

Boost 是一个 C++ 开发者集结的社群，也是一个可自由下载的 C++ 程序库群。它的网址是 <http://boost.org>。现在你应该把它设为你的桌面书签之一。

当然，世上多得是 C++ 组织和网站，但 Boost 有两件事是其他任何组织无可匹敌的。第一，它和 C++ 标准委员会之间有着独一无二的密切关系，并且对委员会深具影响力。Boost 由委员会成员创设，因此 Boost 成员和委员会成员有很大的重叠。Boost 有个目标：作为一个“可被加入标准 C++ 之各种功能”的测试场。这层关系造就的结果是，以 TR1（见条款 54）提案进入标准 C++ 的 14 个新程序库中，超过三分之二奠基于 Boost 的工作成果。

Boost 的第二个特点是：它接纳程序库的过程。它以公开进行的同僚复审（public peer review）为基础。如果你打算贡献一个程序库给 Boost，首先要对 Boost 开发者电邮名单（mailing list）投递作品，让他们评估这个程序库的重要性，并启动初步审查程序。然后开始这个网站所谓的“讨论、琢磨、再次提交”循环周期，直到一切都获得满足为止。

最后，你准备好你的程序库，要正式提交了。会有一位复审管理员出面确认你的程序库符合 Boost 最低要求。例如它必须通过至少两个编译器（以展现至此仍还微不足道的可移植性），你必须证明你的程序库在一个可接受的授权许可下是可用

的（例如这个程序库必须允许免费商业化和非商业化用途）。然后你的提交正式进入 Boost 社群，等待官方复审。复审期间会有志愿者察看你的程序库各种素材（例如源码、设计文档、使用说明等等），并考虑诸如此类的问题：

- 这一份设计和实现有多好？
- 这些代码可跨编译器和操作系统吗？
- 这个程序库有可能被它所设定的目标用户——也就是在这个程序库企图解决问题的领域中工作的人们——使用吗？
- 文档是否清楚、齐备，而且精确？

所有批注都会被投寄至一份 Boost 邮件列表，所以复审者和其他人可以看到并响应其他人的评论。复审最后周期结束之后，复审管理员便表决你的程序库被接受、被有条件接受，或被拒绝。

同僚复审对于阻挡低劣的程序库很有贡献，同时也教育程序库作者认真考虑一个工业强度、跨平台的程序库的设计、实现和文档工程。许多程序库在被 Boost 接受之前，往往经历了一次以上的官方复审。

Boost 内含数十个程序库，而且还不断有更多添加进来。偶尔也会有程序库被从中移除，通常那是因为它们的机能已被新程序库取代，而新程序库提供了更多、更好的机能，或更好的设计（例如更弹性或更有效率）。

Boost 各程序库之间的大小和作用范围有很大变化。举一个极端例子，某些程序库概念上只需数行代码（但在加入错误处理和可移植性后往往变长很多）。例如 **Conversion** 程序库，提供较安全或较方便的转型操作符，其 `numeric_cast` 函数在将数值从某类型转换为另一类型而导致溢出（overflow）或下溢（underflow）或类似问题时会抛出异常。`lexical_cast` 则使我们得以将任何类型（只要支持 `operator<<`）转换为字符串，对程序的诊断和运转志记（logging）都十分有用。另一个极端例子是某些程序库提供大面积能力，甚至可以写成一整本书，这类程序库包括 **Boost Graph Library**（用于编写任意 graph 结构）和 **Boost MPL Library**（一个元编程程序库，`metaprogramming library`）。

Boost 程序库对付的主题非常繁多, 区分数十个类目, 包括:

- **字符串与文本处理**, 覆盖具备类型安全 (type-safe) 的 printf-like 格式化动作、正则表达式 (此为 TR1 同类机能的基础, 见条款 54), 以及语汇单元切割 (tokenizing) 和解析 (parsing)。
- **容器**, 覆盖“接口与 STL 相似且大小固定”的数组 (见条款 54)、大小可变的 bitsets 以及多维数组。
- **函数对象和高级编程**, 覆盖若干被用来作为 TR1 机能基础的程序库。其中一个有趣的程序库是 Lambda, 它让我们得以轻松地随时随地创建函数对象, 但是你却很有可能不太了解你正在做什么:

```
using namespace boost::lambda;    //让 boost::lambda 的机能曝光
std::vector<int> v;
...
std::for_each(v.begin(), v.end(),    //针对 v 内的每一个元素 x,
              std::cout << _1 * 2 + 10 << "\n"); //印出 x * 2+10;
                                              //其中 "_1" 是 Lambda 程序库
                                              //针对当前元素的一个
                                              //占位符号 (placeholder)
```

- **泛型编程 (Generic programming)**, 覆盖一大组 traits classes。关于 traits 请见条款 47。
- **模板元编程 (Template metaprogramming, TMP, 见条款 48)**, 覆盖一个针对编译期 assertions 而写的程序库, 以及 Boost MPL 程序库。MPL 提供了极好的东西, 其中支持编译期实体 (compile-time entities) 诸如 types 的 STL-like 数据结构, 等等。

```
//创建一个 list-like 编译期容器, 其中收纳三个类型:
// (float, double, long double), 并将此容器命名为 "floats"
typedef boost::mpl::list<float, double, long double> floats;
//再创建一个编译期间用以收纳类型的 list, 以 "floats" 内的类型为基础,
//最前面再加上 "int"。新容器取名为 "types"。
typedef boost::mpl::push_front<floats, int>::type types;
```

这样的“类型容器”(常被称为 *typelists*——虽然它们也可以以一个 `mpl::vector` 或 `mpl::list` 为基础) 开启了一扇大门, 通往大范围、火力强大且重要的 TMP 应用程序。

- **数学和数值 (Math and numerics)**, 包括有理数、八元数和四元数 (octonions and quaternions)、常见的公约数 (divisor) 和少见的多重运算、随机数 (又一个影

响 TR1 内部相关机能的程序库)。

- **正确性与测试 (Correctness and testing)**, 覆盖用来将隐式模板接口 (implicit template interfaces, 见条款 41) 形式化的程序库, 以及针对“测试优先”编程形态而设计的措施。
- **数据结构**, 覆盖类型安全 (type-safe) 的 unions (存储各具差异之“任何”类型), 以及 tuple 程序库 (它是 TR1 同类机能的基础)。
- **语言间的支持 (Inter-language support)**, 包括允许 C++ 和 Python 之间的无缝互操作性 (seamless interoperability)。
- **内存**, 覆盖 Pool 程序库, 用来做出高效率而区块大小固定的分配器 (见条款 50), 以及多变化的智能指针 (smart pointers, 见条款 13), 包括 (但不仅仅是) TR1 智能指针。另有一个 non-TR1 智能指针是 `scoped_array`, 那是个 `auto_ptr-like` 智能指针, 用来动态分配数组; 条款 44 曾经示范其用法。
- **杂项**, 包括 CRC 检验、日期和时间的处理、在文件系统上来回移动等等。

请记住, 这只是可在 Boost 中找到的程序库抽样, 不是一份详尽清单。

Boost 提供的程序库可以做很多很多事, 但它并未覆盖整套编程风光。例如其中就没有针对 GUI 开发而设计的程序库, 也没有用以连通数据库的程序库——至少在我下笔此刻没有。然而当你阅读本书时就有了也说不定。到底有没有, 唯一可以确定的办法是常常上网检核。我建议你现在就去访问: <http://boost.org>。纵使你没能找到刚好符合需求的作品, 也一定会在其中发现一些有趣的东西。

请记住

- Boost 是一个社群, 也是一个网站。致力于免费、源码开放、同僚复审的 C++ 程序库开发。Boost 在 C++ 标准化过程中扮演深具影响力的角色。
- Boost 提供许多 TR1 组件实现品, 以及其他许多程序库。

A

本书之外

Beyond Effective C++

《Effective C++》一书覆盖我认为对于以编程为业的 C++ 程序员最重要的一般性准则。如果你有兴趣更强化各种高效做法,我鼓励你试试我的另外两本书:《More Effective C++》和《Effective STL》。

《More Effective C++》覆盖了另一些编程准则,以及对于效能和异常的广泛论述。它也描述重要的 C++ 编程技术如智能指针(smart pointers)、引用计数(reference counting)和代理对象(proxy objects)等等。

《Effective STL》是一本和《Effective C++》一样的准则导向(guideline-oriented)书籍,专注于对 STL (Standard Template Library, 标准模板库)的高效运用。

两本书的目录摘要于下。

《More Effective C++》目录

Basics

- Item 01: Distinguish between pointers and references
- Item 02: Prefer C++-style casts
- Item 03: Never treat arrays polymorphically
- Item 04: Avoid gratuitous default constructors

Operators

- Item 05: Be wary of user-defined conversion functions
- Item 06: Distinguish between prefix and postfix forms of increment and decrement operators
- Item 07: Never overload &&, ||, or ,
- Item 08: Understand the different meanings of new and delete

Exceptions

- Item 09: Use destructors to prevent resource leaks
- Item 10: Prevent resource leaks in constructors
- Item 11: Prevent exceptions from leaving destructors
- Item 12: Understand how throwing an exception differs from passing a parameter or calling a virtual function
- Item 13: Catch exceptions by reference
- Item 14: Use exception specifications judiciously
- Item 15: Understand the costs of exception handling

Efficiency

- Item 16: Remember the 80-20 rule
- Item 17: Consider using lazy evaluation
- Item 18: Amortize the cost of expected computations
- Item 19: Understand the origin of temporary objects
- Item 20: Facilitate the return value optimization
- Item 21: Overload to avoid implicit type conversions
- Item 22: Consider using `op=` instead of stand-alone `op`
- Item 23: Consider alternative libraries
- Item 24: Understand the costs of virtual functions, multiple inheritance, virtual base classes, and RTTI

Techniques

- Item 25: Virtualizing constructors and non-member functions
- Item 26: Limiting the number of objects of a class
- Item 27: Requiring or prohibiting heap-based objects
- Item 28: Smart pointers
- Item 29: Reference counting
- Item 30: Proxy classes
- Item 31: Making functions virtual with respect to more than one object

Miscellany

- Item 32: Program in the future tense
- Item 33: Make non-leaf classes abstract
- Item 34: Understand how to combine C++ and C in the same program
- Item 35: Familiarize yourself with the language standard

《Effective STL》目录

Chapter 1: Containers

- Item 01: Choose your containers with care.
- Item 02: Beware the illusion of container-independent code.
- Item 03: Make copying cheap and correct for objects in containers.
- Item 04: Call `empty` instead of checking `size()` against zero.
- Item 05: Prefer range member functions to their single-element counterparts.
- Item 06: Be alert for C++'s most vexing parse.
- Item 07: When using containers of newed pointers, remember to delete the pointers before the container is destroyed.
- Item 08: Never create containers of `auto_ptr`.
- Item 09: Choose carefully among erasing options.
- Item 10: Be aware of allocator conventions and restrictions.
- Item 11: Understand the legitimate uses of custom allocators.
- Item 12: Have realistic expectations about the thread safety of STL containers.

Chapter 2: vector and string

- Item 13: Prefer vector and string to dynamically allocated arrays.
- Item 14: Use `reserve` to avoid unnecessary reallocations.
- Item 15: Be aware of variations in string implementations.
- Item 16: Know how to pass vector and string data to legacy APIs.
- Item 17: Use “the swap trick” to trim excess capacity.
- Item 18: Avoid using `vector<bool>`.

Chapter 3: Associative Containers

- Item 19: Understand the difference between equality and equivalence.
- Item 20: Specify comparison types for associative containers of pointers.
- Item 21: Always have comparison functions return false for equal values.
- Item 22: Avoid in-place key modification in set and multiset.
- Item 23: Consider replacing associative containers with sorted vectors.
- Item 24: Choose carefully between `map::operator[]` and `map::insert` when efficiency is important.
- Item 25: Familiarize yourself with the nonstandard hashed containers.

Chapter 4: Iterators

- Item 26: Prefer iterator to `const_iterator`, `reverse_iterator`, and `const_reverse_iterator`.
- Item 27: Use `distance` and `advance` to convert a container's `const_iterator`s to iterators.
- Item 28: Understand how to use a `reverse_iterator`'s base iterator.
- Item 29: Consider `istreambuf_iterator`s for character-by-character input.

Chapter 5: Algorithms

- Item 30: Make sure destination ranges are big enough.
- Item 31: Know your sorting options.
- Item 32: Follow remove-like algorithms by `erase` if you really want to remove something.
- Item 33: Be wary of remove-like algorithms on containers of pointers.
- Item 34: Note which algorithms expect sorted ranges.
- Item 35: Implement simple case-insensitive string comparisons via `mismatch` or `lexicographical_compare`.
- Item 36: Understand the proper implementation of `copy_if`.
- Item 37: Use `accumulate` or `for_each` to summarize ranges.

Chapter 6: Functors, Functor Classes, Functions, etc.

- Item 38: Design functor classes for pass-by-value.
- Item 39: Make predicates pure functions.
- Item 40: Make functor classes adaptable.
- Item 41: Understand the reasons for `ptr_fun`, `mem_fun`, and `mem_fun_ref`.
- Item 42: Make sure `less<T>` means operator<.

Chapter 7: Programming with the STL

- Item 43: Prefer algorithm calls to hand-written loops.
- Item 44: Prefer member functions to algorithms with the same names.
- Item 45: Distinguish among `count`, `find`, `binary_search`, `lower_bound`, `upper_bound`, and `equal_range`.
- Item 46: Consider function objects instead of functions as algorithm parameters.
- Item 47: Avoid producing write-only code.
- Item 48: Always `#include` the proper headers.
- Item 49: Learn to decipher STL-related compiler diagnostics.
- Item 50: Familiarize yourself with STL-related web sites.

B

新旧版条款对照

Item Mappings Between Second and Third Editions

《Effective C++》第三版和先前的第二版之间有许多不同，最重要的是它覆盖了许多新信息。第二版内容大多继续存在于第三版中，不过往往以修改过的形式和位置出现。下页表格列出第二版条款内的信息可在第三版哪里找到，下下页表格则是相反方向的对应。

以下表格所列的是信息的对应，不是文字的对应。例如第二版条款 39 “避免在继承体系中做向下转型动作”的观念被移到第三版的**条款 27**，并赋予崭新的文字和示例。更极端的例子是第二版条款 18 “努力让 class 接口完满且最小化”。这个条款的主要结论是，如果函数不需特别访问 class 的 non-public 成分，它通常应该被设计为一个 non-members。第三版中相同的结论却是藉由不同（更强烈）的理由触发，因此第二版的条款 18 对应至第三版的**条款 23**，尽管这两个条款之间的唯一共同点只是它们的结论。

第二版映射至第三版

第二版	第三版	第二版	第三版	第二版	第三版
1	2	18	23	35	32
2	—	19	24	36	34
3	—	20	22	37	36
4	—	21	3	38	37
5	16	22	20	39	27
6	13	23	21	40	38
7	49	24	—	41	41
8	51	25	—	42	39,44
9	52	26	—	43	40
10	50	27	6	44	—
11	14	28	—	45	5
12	4	29	28	46	18
13	4	30	28	47	4
14	7	31	21	48	53
15	10	32	26	49	54
16	12	33	30	50	—
17	11	34	31		